

Hierarchical Planning in BDI Agent Programming Languages

A Formal Approach

Sebastian Sardina

Department of Computer Science and Information Technology
RMIT University
Melbourne, AUSTRALIA

Joint work with Lavindra de Silva and Lin Padgham

June 29, 2006

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning
- 5 Conclusions

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning
- 5 Conclusions

Motivation

BDI-style **agent-oriented programming** is a novel approach to programming complex systems in highly dynamic environments.

These systems are very open to changes in the environment, flexible, and robust.

However, they usually lack any **lookahead planning** capabilities.

HTN planning is a well-known practical approach to planning.

Incorporate HTN planning into current BDI programming frameworks with:
(i) a clear semantics; (ii) a direct implementation.

Motivation

BDI-style **agent-oriented programming** is a novel approach to programming complex systems in highly dynamic environments.

These systems are very open to changes in the environment, flexible, and robust.

However, they usually lack any **lookahead planning** capabilities.

HTN planning is a well-known practical approach to planning.

Incorporate HTN planning into current BDI programming frameworks with:
(i) a clear semantics; (ii) a direct implementation.

Motivation

BDI-style **agent-oriented programming** is a novel approach to programming complex systems in highly dynamic environments.

These systems are very open to changes in the environment, flexible, and robust.

However, they usually lack any **lookahead planning** capabilities.

HTN planning is a well-known practical approach to planning.

Incorporate HTN planning into current BDI programming frameworks with:
(i) a clear semantics; (ii) a direct implementation.

Motivation

BDI-style **agent-oriented programming** is a novel approach to programming complex systems in highly dynamic environments.

These systems are very open to changes in the environment, flexible, and robust.

However, they usually lack any **lookahead planning** capabilities.

HTN planning is a well-known practical approach to planning.

Incorporate HTN planning into current BDI programming frameworks with:
(i) a clear semantics; (ii) a direct implementation.

Motivation

BDI-style **agent-oriented programming** is a novel approach to programming complex systems in highly dynamic environments.

These systems are very open to changes in the environment, flexible, and robust.

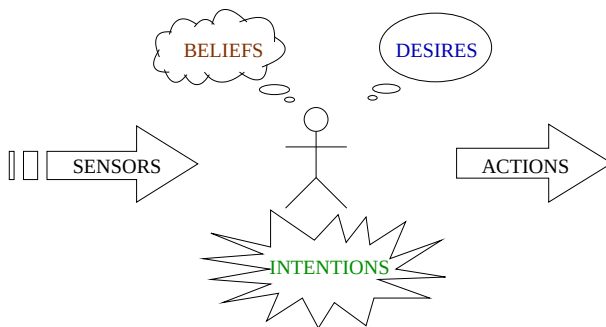
However, they usually lack any **lookahead planning** capabilities.

HTN planning is a well-known practical approach to planning.

Incorporate HTN planning into current BDI programming frameworks with:
(i) a clear semantics; (ii) a direct implementation.

Belief-Desire-Intention (BDI) model of agency

Set of concepts for **thinking about** and **building** agents.



Behavior arises due to the agent **committing** to some of its **desires**, and selecting **actions** that achieve its **intentions** given its **beliefs**.

BDI Agent Programming Languages

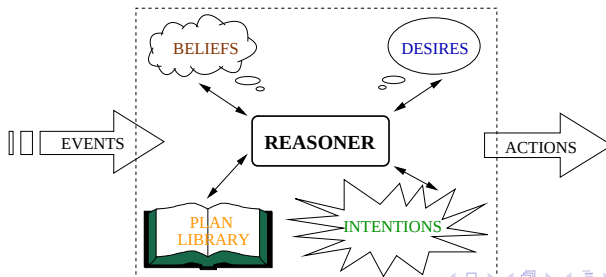
Key features of BDI agent-oriented systems

Beliefs: information about the world.

Events: goals/desires to resolve; internal or external.

Plan library: recipes for handling goals-events: $e : \psi \leftarrow P$.

Intentions: partially uninstantiated programs; commitment.



BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 *Observe the environment for new external events.*
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 Observe the environment for new *external* events.
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 Observe the environment for new *external* events.
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 Observe the environment for new *external* events.
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- ➊ *Observe* the environment for new *external* events.
- ➋ Pick a pending event *e*.
- ➌ Select *relevant* plans from library (match event).
- ➍ Select *applicable* plans from relevant set (match context).
- ➎ If event *e* is *external*, create new intention with selected plan.
- ➏ If event *e* is *internal*, update the corresponding intention.
- ➐ Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- ➑ Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- ➊ *Observe* the environment for new *external* events.
- ➋ Pick a pending event *e*.
- ➌ Select *relevant* plans from library (match event).
- ➍ Select *applicable* plans from relevant set (match context).
- ➎ If event *e* is *external*, create new intention with selected plan.
- ➏ If event *e* is *internal*, update the corresponding intention.
- ➐ Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- ➑ Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 *Observe* the environment for new *external* events.
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 *Partially execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 Repeat cycle.

BDI Rational Architecture of Agency

The BDI Execution Cycle [Rao&Georgeff 92]

- 1 *Observe* the environment for new *external* events.
- 2 Pick a pending event *e*.
- 3 Select *relevant* plans from library (match event).
- 4 Select *applicable* plans from relevant set (match context).
- 5 If event *e* is *external*, create new intention with selected plan.
- 6 If event *e* is *internal*, update the corresponding intention.
- 7 Partially *execute* some intention (may post internal events).
If execution fails, then perform *failure recovery*.
- 8 **Repeat cycle.**

BDI Rational Architecture of Agency (cont.)

Key Points of BDI Programming

- Flexible and responsible to the environment: “**reactive planning.**”
- Well suited for **soft real-time** reasoning and control.
- Relies on **context sensitive** subgoal expansion: “act as you go.”
- Leave for **as late as possible** the choice of which plans to commit to as the chosen course of action to achieve (sub)goals.
- **Modular** programming.
- Nondeterminism on **choosing** plans and bindings.
- **No mechanism for doing lookahead for solving choices**
Generally programmed *explicitly* by the BDI programmer.

BDI Rational Architecture of Agency (cont.)

Key Points of BDI Programming

- Flexible and responsible to the environment: “**reactive planning.**”
- Well suited for **soft real-time** reasoning and control.
- Relies on **context sensitive** subgoal expansion: “act as you go.”
- Leave for **as late as possible** the choice of which plans to commit to as the chosen course of action to achieve (sub)goals.
- **Modular** programming.
- Nondeterminism on **choosing** plans and bindings.
- **No mechanism for doing lookahead for solving choices**
Generally programmed *explicitly* by the BDI programmer.

Why do we want planning?

Some reasons for lookahead capabilities for BDI agents:

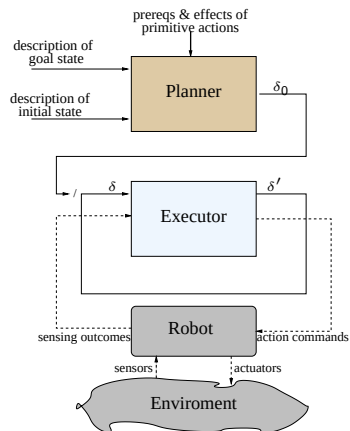
- Important resources may be used in taking actions that do not lead to successful outcome (e.g., fuel).
- Actions may sometimes not be reversible (e.g., break glass).
- Execution of actions may take substantial more time than just “thinking” ahead (e.g., travelling to a distant location).
- Actions’ execution may include undesirable side-effects (e.g., email notifications).

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning
- 5 Conclusions

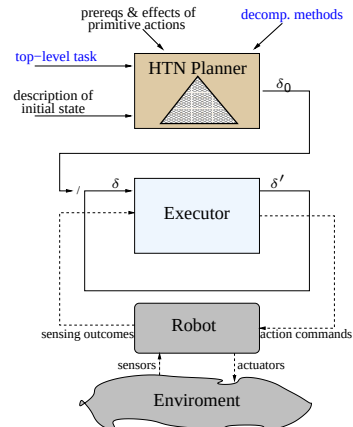
Classical Planning [SRI Shakey “The Robot” ’70]

- Planing from **first-principles**.
- **Goals-to-be**: *achieve* ϕ ($At(rmit)$).
- Input to planner:
 - action descriptions;
 - initial state;
 - goal state.
- Output of planner:
 - a plan that achieves the goal.

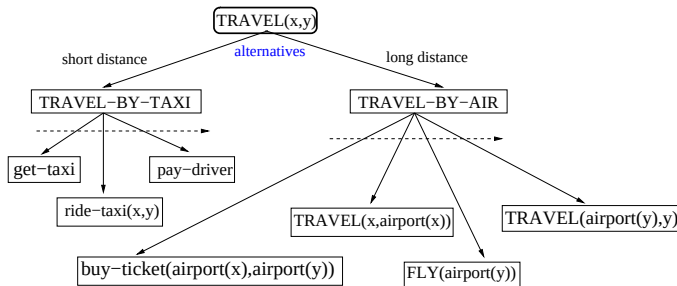


Hierarchical Planning

- Planing by **task reduction/decomposition**.
- **Goals-to-do**: *travelTo(loc)*.
- Includes domain procedural knowledge: captures domain hierarchical structure.
- Input to planner:
 - action descriptions;
 - initial state;
 - *top-level task*;
 - *decomposition methods*.
- Output of planner:
 - a plan that achieves the goal.



HTN Planning: An Example



State: set of atoms: $At(loc)$.

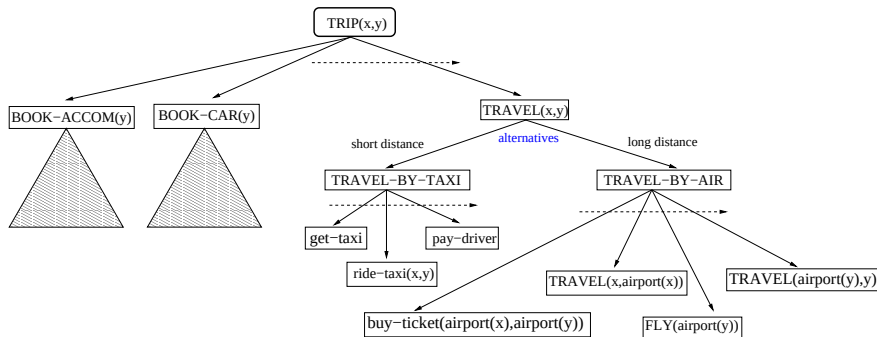
Tasks: *primitive* or *compound*.

Task Network: set of tasks T + order/state constraints ϕ .

Method: a way to solve a compound task e using a network d .

Plan: a sequence of primitive tasks.

HTN Planning: An Example (cont.)



Components of HTN Planning

- **State**: set of atoms + CWA
- **Task**: a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$** :
 - set of tasks T + order/state constraints ϕ .
 - $(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$** : a way of solving high-level task e .
 - $[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ** : sequence of primitive tasks.

Components of HTN Planning

- **State:** set of atoms + CWA
- **Task:** a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network** $d = (T, \phi)$:
 - set of tasks T + order/state constraints ϕ .
 - $(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method** $m = [e, \psi, d]$: a way of solving high-level task e .
 - $[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan** σ : sequence of primitive tasks.

Components of HTN Planning

- **State:** set of atoms + CWA
- **Task:** a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$:**

set of tasks T + order/state constraints ϕ .

$(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$:** a way of solving high-level task e .

$[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ :** sequence of primitive tasks.

Components of HTN Planning

- **State:** set of atoms + CWA
- **Task:** a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$:**

set of tasks T + order/state constraints ϕ .

$(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$:** a way of solving high-level task e .

$[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ :** sequence of primitive tasks.

Components of HTN Planning

- **State**: set of atoms + CWA
- **Task**: a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$** :
 set of tasks T + order/state constraints ϕ .
 $(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$** : a way of solving high-level task e .
 $[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ** : sequence of primitive tasks.

Components of HTN Planning

- **State**: set of atoms + CWA
- **Task**: a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$** :
 - set of tasks T + order/state constraints ϕ .
 - $(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$** : a way of solving high-level task e .
 - $[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 - $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ** : sequence of primitive tasks.

Components of HTN Planning

- **State:** set of atoms + CWA
- **Task:** a common goal that the agent may need to achieve.
 - Primitive task a : with precondition and effects: $pickObj(o)$
 - High-level task e : cannot be directly executed: $deliverObj(o, loc)$
- **Task Network $d = (T, \phi)$:**

set of tasks T + order/state constraints ϕ .

$(\{t_1 = pickObj(2), t_2 = pickObj(1), t_3 = dispose(2)\}, (t_1 < t_3))$
- **Method $m = [e, \psi, d]$:** a way of solving high-level task e .

$[deliverObj(o, loc), Serve(loc), (\{t = inCityDeliver(o, oloc, loc)\},$
 $(At(o, oloc), t) \wedge (inCity(oloc, c), t) \wedge (inCity(loc, c), t))]$
- **Plan σ :** sequence of primitive tasks.

HTN Planning: Formalization

Formal Specification [EHN94a, EHN94b]

- An HTN planning domain is a pair $\mathcal{D} = (\Pi, \Lambda)$:
Method library Π ; action description Λ .
- An HTN problem is a triple $\mathbf{P} = \langle d, \mathcal{B}, \mathcal{D} \rangle$
Solve task network d in state $\mathcal{B}$ using domain $\mathcal{D}$.
- An HTN planning solution for problem $\mathbf{P}$ is a plan σ that stands for a **full decomposition** of task network d .
Operational and model-theoretic semantics.

HTN Planning: Formalization

Formal Specification [EHN94a, EHN94b]

- An HTN planning domain is a pair $\mathcal{D} = (\Pi, \Lambda)$:
Method library Π ; action description Λ .
- An HTN problem is a triple $\mathbf{P} = \langle d, \mathcal{B}, \mathcal{D} \rangle$
Solve task network d in state $\mathcal{B}$ using domain $\mathcal{D}$.
- An HTN planning solution for problem $\mathbf{P}$ is a plan σ that stands for a **full decomposition** of task network d .
Operational and model-theoretic semantics.

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning
- 5 Conclusions

The CAN Language [WPHT02]

A CAN agent is defined as $Agt = \langle \mathcal{N}, \Pi, \mathcal{G}, \mathcal{B}, \mathcal{A}, \Gamma \rangle$, where:

- $\mathcal{N}$ is the **agent name**.
- $\mathcal{G}$ is the **goal base**: for conflict resolution.
- $\mathcal{B}$ is the **belief base**: current agent's knowledge.
- $\mathcal{A}$ is the sequence of actions executed.
- Π is a **plan library** containing plan rules $e : \psi \leftarrow P$
 e is the triggering event and ψ is the context condition
 P is the plan-body from the following language:

$$P ::= nil \mid act \mid ?\phi \mid +b \mid -b \mid P_1; P_2 \mid P_1 \parallel P_2 \mid !e \mid \underline{Goal(\phi_s, P_1, \phi_f)}$$
- Γ is the **intention base**: partially uninstantiated plan-bodies.

Goal-programs: $\text{Goal}(\phi_s, P, \phi_f)$

Achieve goal ϕ_s by using plan P ; failing if ϕ_f becomes true.

Example

$\text{Goal}(\neg \text{Hungry}, !\text{satisfyHunger}, \neg \text{foodAvailable})$.

$\text{Goal}(\phi_s, P, \phi_f)$ Properties

- 1 If ϕ_s becomes true, terminate successfully.
- 2 If ϕ_f becomes true, terminate with failure.
- 3 If P terminates successfully and ϕ_s is still *not* true, P is re-tried.
- 4 If P fails and ϕ_s is still *not* true, P is re-tried.

The CAN Language: Operational Semantics

Defined in two levels:

- 1 Basic configurations: $\langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle$
- 2 Agent level execution: $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', \Gamma' \rangle$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{step}$$

$$\frac{e \text{ is a new external event}}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{!e\} \rangle} A_{event}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \not\longrightarrow}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \setminus \{P\} \rangle} A_{clean}$$

The CAN Language: Operational Semantics

Defined in two levels:

- 1 Basic configurations: $\langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle$
- 2 Agent level execution: $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', \Gamma' \rangle$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{step}$$

$$\frac{e \text{ is a new external event}}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{!e\} \rangle} A_{event}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \not\longrightarrow}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \setminus \{P\} \rangle} A_{clean}$$

The CAN Language: Operational Semantics

Defined in two levels:

- 1 Basic configurations: $\langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle$
- 2 Agent level execution: $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', \Gamma' \rangle$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{step}$$

$$\frac{e \text{ is a new external event}}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{!e\} \rangle} A_{event}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \not\longrightarrow}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \setminus \{P\} \rangle} A_{clean}$$

The CAN Language: Operational Semantics

Defined in two levels:

- ① Basic configurations: $\langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle$
- ② Agent level execution: $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', \Gamma' \rangle$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{step}$$

$$\frac{\text{e is a new external event}}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{!e\} \rangle} A_{event}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \not\longrightarrow}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \setminus \{P\} \rangle} A_{clean}$$

The CAN Language: Operational Semantics

Defined in two levels:

- ① Basic configurations: $\langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle$
- ② Agent level execution: $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', \Gamma' \rangle$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \longrightarrow \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{step}$$

$$\frac{e \text{ is a new external event}}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{!e\} \rangle} A_{event}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \not\longrightarrow}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \setminus \{P\} \rangle} A_{clean}$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow \langle \psi_1 : P_1, \dots, \psi_n : P_n \rangle \longrightarrow$$

$$P_i \triangleright \langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle \xrightarrow{*}$$

$$\langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle$$

$$\frac{\Delta = \{ \psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e') \}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow (\psi_1 : P_1, \dots, \psi_n : P_n) \longrightarrow$$

$$P_i \triangleright (\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n) \xrightarrow{*}$$

$$(\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n)$$

$$\frac{\Delta = \{\psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e')\}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$!e \longrightarrow (\psi_1 : P_1, \dots, \psi_n : P_n) \longrightarrow$

$P_i \triangleright (\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n) \xrightarrow{*}$
 $(\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n)$

$$\frac{\Delta = \{\psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e')\}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow (\psi_1 : P_1, \dots, \psi_n : P_n) \longrightarrow$$

$$P_i \triangleright (\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n) \xrightarrow{*}$$

$$(\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n)$$

$$\frac{\Delta = \{\psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e')\}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow (\psi_1 : P_1, \dots, \psi_n : P_n) \longrightarrow$$

$$P_i \triangleright (\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n) \xrightarrow{*}$$

$$(\psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n)$$

$$\frac{\Delta = \{\psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e')\}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow \langle \psi_1 : P_1, \dots, \psi_n : P_n \rangle \longrightarrow$$

$$P_i \triangleright \langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle \xrightarrow{*}$$

$$\langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle$$

$$\frac{\Delta = \{ \psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e') \}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\vdash}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow \langle \psi_1 : P_1, \dots, \psi_n : P_n \rangle \longrightarrow$$

$$P_i \triangleright \langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle \xrightarrow{*}$$

$$\langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle$$

$$\frac{\Delta = \{ \psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e') \}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language: Operational Semantics (cont.)

$$!e \longrightarrow \langle \psi_1 : P_1, \dots, \psi_n : P_n \rangle \longrightarrow$$

$$P_i \triangleright \langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle \xrightarrow{*}$$

$$\langle \psi_1 : P_1, \dots, \psi_{i-1} : P_{i-1}, \psi_{i+1} : P_{i+1}, \dots, \psi_n : P_n \rangle$$

$$\frac{\Delta = \{ \psi_i \theta : P_i \theta \mid e' : \psi_i \leftarrow P_i \in \Pi \wedge \theta = \text{mgu}(e, e') \}}{\langle \mathcal{B}, \mathcal{A}, !e \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle} \text{Event}$$

$$\frac{\psi_i : P_i \in \Delta \quad \mathcal{B} \models \psi_i \theta}{\langle \mathcal{B}, \mathcal{A}, \langle \Delta \rangle \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_i \theta \triangleright \langle \Delta \setminus \psi_i : P_i \rangle \rangle} \text{Sel}$$

$$\frac{}{\langle \mathcal{B}, \mathcal{A}, (nil \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, nil \rangle} \triangleright_t \frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \not\rightarrow}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \longrightarrow \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f$$

The CAN Language

However, CAN does not include any planning mechanism.

Our objective: add lookahead capabilities to CAN in a way that:

- 1 the original semantics can be extended;
- 2 an implementation can be easily produced.

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning**
- 5 Conclusions

BDI Systems and HTN Planners: Similarities

BDI Systems	HTN Systems
belief base	state
plan library	method library
event	high-level task
action	primitive task
plan-body/program	network task
plan rule	method
plan rule context	method precondition
test ? <i>p</i> in plan-body	state constraints
sequence in plan-body	ordering constraint $\prec$
parallelism in plan-body	no ordering constraint

The CANPLAN Language: Operational Semantics

We distinguish two types of transitions:

- $\xrightarrow{\text{bdi}}$: transitions for the BDI execution cycle.
- $\xrightarrow{\text{plan}}$: transitions within a planning context.

New construct $\text{Plan}(P)$: lookahead on plan P .

$$\frac{\langle B, A, P \rangle \xrightarrow{\text{plan}} \langle B', A', P' \rangle \quad \langle B', A', P' \rangle \xrightarrow{\text{plan}_*} \langle B'', A'', \text{nil} \rangle}{\langle B, A, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}} \langle B', A', \text{Plan}(P') \rangle} \text{Plan}$$

OBS: inspired by the semantics of Σ in [De Giacomo&Levesque 99]

The CANPLAN Language: Operational Semantics

We distinguish two types of transitions:

- $\xrightarrow{\text{bdi}}$: transitions for the BDI execution cycle.
- $\xrightarrow{\text{plan}}$: transitions within a planning context.

New construct $\text{Plan}(P)$: lookahead on plan P .

$$\frac{\langle \mathcal{B}, \mathcal{A}, P \rangle \xrightarrow{\text{plan}} \langle \mathcal{B}', \mathcal{A}', P' \rangle \quad \langle \mathcal{B}', \mathcal{A}', P' \rangle \xrightarrow{\text{plan}_*} \langle \mathcal{B}'', \mathcal{A}'', \text{nil} \rangle}{\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}} \langle \mathcal{B}', \mathcal{A}', \text{Plan}(P') \rangle} \text{Plan}$$

OBS: inspired by the semantics of Σ in [De Giacomo&Levesque 99]

The CANPLAN Language: Operational Semantics (cont.)

We also need to:

- allow retry of alternatives *only* in the BDI execution cycle;
- perform agent steps based on bdi-type basic transitions.

$$\frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \xrightarrow{\text{bdi}} \quad}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \xrightarrow{\text{bdi}} \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f^{\text{bdi}}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \xrightarrow{\text{bdi}} \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{\text{step}}$$

The CANPLAN Language: Operational Semantics (cont.)

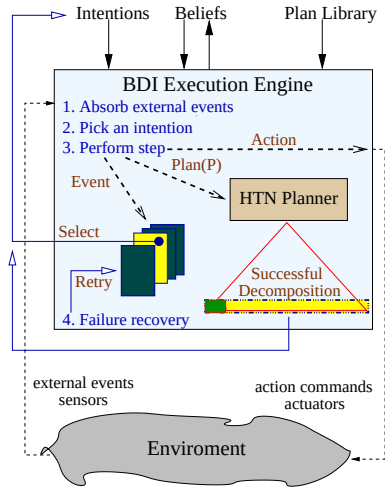
We also need to:

- allow retry of alternatives *only* in the BDI execution cycle;
- perform agent steps based on bdi-type basic transitions.

$$\frac{\langle \mathcal{B}, \mathcal{A}, P_1 \rangle \xrightarrow{\text{bdi}} \quad}{\langle \mathcal{B}, \mathcal{A}, (P_1 \triangleright P_2) \rangle \xrightarrow{\text{bdi}} \langle \mathcal{B}, \mathcal{A}, P_2 \rangle} \triangleright_f^{\text{bdi}}$$

$$\frac{P \in \Gamma \quad \langle \mathcal{B}, \mathcal{A}, P \rangle \xrightarrow{\text{bdi}} \langle \mathcal{B}', \mathcal{A}', P' \rangle}{\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \rangle \Longrightarrow \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}', \mathcal{A}', (\Gamma \setminus \{P\}) \cup \{P'\} \rangle} A_{\text{step}}$$

CANPLAN Architecture



Main Results About $\text{Plan}(P)$

Some remarks about Plan

- 1 Plan is built-in within the BDI execution cycle.
- 2 Plan's semantics is defined “in terms” of the BDI cycle.
- 3 But failure-handling is left to the BDI cycle only.

Theorem

*An intention $\text{Plan}(P)$ **never fails** in an environment-free execution.*

Main Results About $\text{Plan}(P)$

Some remarks about Plan

- 1 Plan is built-in within the BDI execution cycle.
- 2 Plan's semantics is defined “in terms” of the BDI cycle.
- 3 But failure-handling is left to the BDI cycle only.

Theorem

*An intention $\text{Plan}(P)$ **never fails** in an environment-free execution.*

Main Results About $\text{Plan}(P)$ (cont.)

Theorem

*All executions of $\text{Plan}(P)$ correspond to **HTN solutions** of P w.r.t. the agent's plan-library Π (and vice-versa).*

Basis for an implementation: JACK+ JSHOP [Lavindra&Padgham 05]

- JACK is a BDI-style programming language similar to CAN;
- JSHOP is an Java-based HTN planner implementation.

(Obs.: no concurrency || and no goal-programs Goal, yet)

Planning for Declarative Goals: Use P for achieving ϕ_s

- (A) P is used towards the eventual satisfaction of goal ϕ_s .
- (B) Allow partial executions of P achieving ϕ_s .
- (C) Commitment on the goal ϕ_s .
- (D) Goal dropping mechanism.
- (E) P may be solved partially.

ALTERNATIVES	A	B	C	D	E
$\text{Plan}(P; ?\phi_s)$	✓				
$\text{Plan}(\text{Goal}(\phi_s, P, \phi_f))$	✓	✓		✓	
$\text{Goal}(\phi_s, \text{Plan}(P), \phi_f)$		✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(P; ?\phi_s), \phi_f)$	✓	✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$	✓	✓	✓	✓	✓

$$\text{Plan}(\phi_s, P, \phi_f) \stackrel{\text{def}}{=} \text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$$

Planning for Declarative Goals: Use P for achieving ϕ_s

- (A) P is used towards the eventual satisfaction of goal ϕ_s .
- (B) Allow partial executions of P achieving ϕ_s .
- (C) Commitment on the goal ϕ_s .
- (D) Goal dropping mechanism.
- (E) P may be solved partially.

ALTERNATIVES	A	B	C	D	E
$\text{Plan}(P; ?\phi_s)$	✓				
$\text{Plan}(\text{Goal}(\phi_s, P, \phi_f))$	✓	✓		✓	
$\text{Goal}(\phi_s, \text{Plan}(P), \phi_f)$		✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(P; ?\phi_s), \phi_f)$	✓	✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$	✓	✓	✓	✓	✓

$$\text{Plan}(\phi_s, P, \phi_f) \stackrel{\text{def}}{=} \text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$$

Planning for Declarative Goals: Use P for achieving ϕ_s

- (A) P is used towards the eventual satisfaction of goal ϕ_s .
- (B) Allow partial executions of P achieving ϕ_s .
- (C) Commitment on the goal ϕ_s .
- (D) Goal dropping mechanism.
- (E) P may be solved partially.

ALTERNATIVES	A	B	C	D	E
$\text{Plan}(P; ?\phi_s)$	✓				
$\text{Plan}(\text{Goal}(\phi_s, P, \phi_f))$	✓	✓		✓	
$\text{Goal}(\phi_s, \text{Plan}(P), \phi_f)$		✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(P; ?\phi_s), \phi_f)$	✓	✓	✓	✓	
$\text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$	✓	✓	✓	✓	✓

$$\text{Plan}(\phi_s, P, \phi_f) \stackrel{\text{def}}{=} \text{Goal}(\phi_s, \text{Plan}(\text{Goal}(\phi_s, P, \phi_f)), \phi_f)$$

Outline

- 1 Motivation
- 2 HTN Planning
- 3 The CAN Language
- 4 CANPLAN: CAN + Planning
- 5 Conclusions**

Conclusions

- ➊ Reviewed BDI programming & HTN planning.
⇒ Pointed out strong connections.
- ➋ Added a planning mechanism to BDI languages by:
 - Incorporating on-demand planning via a new language construct.
 - Modifying 4 derivation rules in the original CAN semantics.
- ➌ Showed the correspondence with HTN planning
⇒ A prototype implementation: JACK+JSHOP.
- ➍ Investigated ways of doing planning for declarative goals.

Other Issues and Future Work

- ❶ Planning upon failure (PROPICE-PLAN & CPEF): $P \triangleright \text{Plan}(P')$
- ❷ Using first-principles planning for learning new plans:
When no relevant plan is available $\implies$ classical planning.
Add new plan to the BDI plan library Π .
- ❸ Planning in the context of other intentions.
- ❹ Planning up to some level of abstraction.
Leave details to the BDI execution engine.
- ❺ Use SHOP2 for implementing planning in JACK.

Related Work

- ❶ Formal BDI systems with no planning:
AGENTSpeak, 3APL [HdBvdHM99], dMARS [dKLW98]
- ❷ Systems/architectures mixing planning with BDI execution:
 - SHOP and multi-agent environments: IMPACT [DMN03].
 - Planners that interleave execution: IPeM [AIS88]; SAGE [Kno95].
 - Planning always: RETSINA [PSS⁺99]
 - Planning on failure: PROPICE-PLAN [DI99]; CPEF [WM98]
- ❸ INDIGoLOG [DGL99]: execution+planning in sit-calc
 - Not a typical programming language in the BDI tradition
 - GoLOG and HTN planning: [Gab02]

Agents are not objects!

- ❶ An object does not have control over its behavior.
⇒ *Objects do it for free; agents do it because they want to*
- ❷ Object model says nothing about flexible autonomous behavior.
⇒ Reactive, pro-active, flexible, robust.
- ❸ Standard object model is single-threaded.
⇒ Multiagent systems are inherently multi-threaded.

HTN Planning (cont.)

Formal Specification [EHN94a, EHN94b]

- An HTN planning domain is a pair $\mathcal{D} = (\Pi, \Lambda)$:
Method library Π ; action description Λ .
- An HTN problem is a triple $\mathbf{P} = \langle d, \mathcal{B}, \mathcal{D} \rangle$
Solve task network d in state $\mathcal{B}$ using domain $\mathcal{D}$.
- An HTN planning solution for problem $\mathbf{P}$ is a plan σ that stands for a **full decomposition** of task network d .
Operational and model-theoretic semantics
 $\implies \text{sol}(d, \mathcal{B}, \mathcal{D})$: set of all plans that solve d .

HTN Planning (cont.)

Formal Specification [EHN94a, EHN94b]

- An HTN planning domain is a pair $\mathcal{D} = (\Pi, \Lambda)$:
Method library Π ; action description Λ .
- An HTN problem is a triple $\mathbf{P} = \langle d, \mathcal{B}, \mathcal{D} \rangle$
Solve task network d in state $\mathcal{B}$ using domain $\mathcal{D}$.
- An HTN planning solution for problem $\mathbf{P}$ is a plan σ that stands for a **full decomposition** of task network d .
Operational and model-theoretic semantics
 $\implies \text{sol}(d, \mathcal{B}, \mathcal{D})$: set of all plans that solve d .

Plan(P) Construct Guarantees Execution

Definition (Types of Executions)

Execution: sequence $C_0, \dots, C_n$ such that $C_i \implies C_{i+1}$

Environment-free: if agent derivation rule A_{event} is not used.

Successful intention execution: if $P_n = nil$, where $P_n \in \Gamma_n$.

Theorem

Let $C = \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \{\text{Plan}(P)\} \rangle$ such that $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}}$.
If E is an *environment-free* agent execution of C , then intention $\text{Plan}(P)$ is either *executing* or has been *successfully executed* in E .
Moreover, there is an execution E^s of C in which intention $\text{Plan}(P)$ has been successfully executed in E^s .

Plan(P) Construct Guarantees Execution

Definition (Types of Executions)

Execution: sequence $C_0, \dots, C_n$ such that $C_i \implies C_{i+1}$

Environment-free: if agent derivation rule A_{event} is not used.

Successful intention execution: if $P_n = nil$, where $P_n \in \Gamma_n$.

Theorem

Let $C = \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \{\text{Plan}(P)\} \rangle$ such that $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}}$.
If E is an **environment-free** agent execution of C , then intention $\text{Plan}(P)$ is either **executing** or has been **successfully executed** in E .
Moreover, there is an execution E^s of C in which intention $\text{Plan}(P)$ has been successfully executed in E^s .

Plan(P) is HTN-planning

Theorem

For any bounded agent:

- ① $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}} \text{iff } \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda) \neq \emptyset.$
- ② If $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}_*} \langle \mathcal{B}', \mathcal{A} \cdot \text{act}_1 \cdot \dots \cdot \text{act}_k, \text{Plan}(P') \rangle$ **iff** there exists a plan $\sigma = \underline{\text{act}_1 \cdot \dots \cdot \text{act}_k} \cdot \dots \cdot \text{act}_n \in \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda).$
- ③ If there exists a plan $\sigma = \text{act}_1 \cdot \dots \cdot \text{act}_n \in \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda),$ then $\langle \mathcal{B}, \mathcal{A}, (\text{act}_1; \dots; \text{act}_n) \rangle \xrightarrow{\text{bdi}_*} \langle \mathcal{B}', \mathcal{A} \cdot \sigma, \text{nil} \rangle.$

Basis for an implementation: JACK+ JSHOP [?]
(But: no concurrency || and no goal-programs Goal)

Plan(P) is HTN-planning

Theorem

For any bounded agent:

- ① $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}} \text{iff } \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda) \neq \emptyset.$
- ② If $\langle \mathcal{B}, \mathcal{A}, \text{Plan}(P) \rangle \xrightarrow{\text{bdi}_*} \langle \mathcal{B}', \mathcal{A} \cdot \text{act}_1 \cdot \dots \cdot \text{act}_k, \text{Plan}(P') \rangle$ **iff** there exists a plan $\sigma = \underline{\text{act}_1 \cdot \dots \cdot \text{act}_k} \cdot \dots \cdot \text{act}_n \in \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda).$
- ③ If there exists a plan $\sigma = \text{act}_1 \cdot \dots \cdot \text{act}_n \in \text{sol}(P, \mathcal{B}, \Pi \cup \Lambda),$ then $\langle \mathcal{B}, \mathcal{A}, (\text{act}_1; \dots; \text{act}_n) \rangle \xrightarrow{\text{bdi}_*} \langle \mathcal{B}', \mathcal{A} \cdot \sigma, \text{nil} \rangle.$

Basis for an implementation: JACK+ JSHOP [?]
(But: no concurrency || and no goal-programs Goal)

Planning vs BDI Execution

$\text{CANPLAN}^- = \text{CANPLAN}$ with no $\parallel$ and no Goal

Theorem

Program P simulates program $\text{Plan}(P)$ in every CANPLAN^- agent.

Observations:

- ① CANPLAN : $\text{Plan}(\text{Plan}(P_1) \parallel P_2) = \text{Plan}(P_1 \parallel P_2) \neq \text{Plan}(P_1) \parallel P_2$.
- ② INDIGOLOG : $\Sigma(\Sigma(P_1) \parallel P_2) \neq \Sigma(P_1 \parallel P_2)$.

Theorem

Program P may be successfully executed in $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{P\} \rangle$, but not in $C' = \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{\text{Plan}(P)\} \rangle$.

Planning vs BDI Execution

$\text{CANPLAN}^- = \text{CANPLAN}$ with no $\parallel$ and no Goal

Theorem

Program P simulates program $\text{Plan}(P)$ in every CANPLAN^- agent.

Observations:

- ① CANPLAN : $\text{Plan}(\text{Plan}(P_1) \parallel P_2) = \text{Plan}(P_1 \parallel P_2) \neq \text{Plan}(P_1) \parallel P_2$.
- ② INDIGOLOG : $\Sigma(\Sigma(P_1) \parallel P_2) \neq \Sigma(P_1 \parallel P_2)$.

Theorem

Program P may be successfully executed in $\langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{P\} \rangle$, but not in $C' = \langle \mathcal{N}, \Lambda, \Pi, \mathcal{B}, \mathcal{A}, \Gamma \cup \{\text{Plan}(P)\} \rangle$.

For Further Reading: BDI Theory



Michael Bratman.

Intentions, Plans, and Practical Reason.

Harvard University Press, 1987.



Daniel Dennett.

The Intentional Stance.

MIT Press, 1987.



Philip R. Cohen and Hector J. Levesque.

Intention is choice with commitment.

Artificial Intelligence Journal, 42:213–261, 1990.



A.S. Rao and M.P. Georgeff.

Modeling rational agents within a BDI-architecture.

In *Proceedings of KR-91*, pages 473–484, 1991.

For Further Reading: BDI Formal Languages

-  M. Winikoff, L. Padgham, J. Harland, and J. Thangarajah.
Declarative & procedural goals in intelligent agent systems.
In Proceedings of KR-02, 2002.
-  K. Hindriks, F. de Boer, W. van der Hoek, and J.-J. Meyer.
Agent programming in 3APL.
Autonomous Agents and Multi-Agent Systems, 2:357–401, 1999.
-  M. d'Inverno, D. Kinny, M. Luck, and M. Wooldridge.
A formal specification of dMars.
In Proceedings of ATAL-97, 1998.
-  Anand S. Rao and Michael P. Georgeff.
An abstract architecture for rational agents.
In Proceedings of KR-92, pages 438–449, 1992.

For Further Reading: BDI Systems/Architectures



J. Dix, H. Munoz-Avila, and D. Nau
IMPACTing SHOP: Planning in a Multi-Agent Environment.
Annals of Math and AI, 37(4), pages 381–407, 2003.



J. Ambros-Ingerson and S. Steel.
Integrating planning, execution and monitoring.
In Proceedings of AAAI-88, 1988.



Olivier Despouys and Francois Felix Ingrand.
Propice-plan: Toward a unified framework for planning and execution.
In Proc. of European Conference on Planning, 1999.



M. Paolucci, D. Kalp, A. Pannu, O. Shehory, and K. Sycara.
A Planning Component for RETSINA Agents.
In Proceedings of ATAL-99, pages 147–161, 1999.

For Further Reading: HTN and Sit-calc

-  Kutluhan Erol, James Hendler, and Dana S. Nau.
HTN planning: Complexity and expressivity.
In Proceedings of AAAI-94, pages 1123–1228, 1994.
-  Kutluhan Erol, James Hendler, and Dana S. Nau.
Semantics for hierarchical task-network planning.
Technical report, College Park, MD, USA, 1994.
-  Giuseppe De Giacomo and Hector Levesque.
An incremental interpreter for high-level programs with sensing.
In H. Levesque and F., editors, Logical foundation for cognitive agents: contributions in honor of Ray Reiter, pages 86–102. 1999.
-  Alfredo Gabaldon.
Programming hierarchical task networks in the situation calculus.
In AIPS'02 Workshop on On-line Planning and Scheduling, 2002.